1. Write the python program to solve 8-Puzzle Problem

Reg. No: 192511013

Aim

To write a python program to solve the 8-puzzle problem using the Breadth-first search (BFS) algorithm.

Algorithm

1. Start with the initial state and the goal state.
2. Inert the initial state into a queue.
3. Remove a state from the queue and check whether it matches the goal state.
4. Find the blank tile represented by 0.
5. Move the blank tile up, down, left, or right whenever possible.
6. Add the new states to the queue if they have not been visited.
7. Display the solution path when the goal state is reached.

Program

```
from collections import deque

def solve_puzzle (start, goal):
    queue = deque ([[start, [start]]])
    visited = {tuple (start)}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        State, path = queue.popleft ()
```

Sample Input

plain text

Enter initial state : 1 2 3 4 0 6 7 5 8

Enter goal state : 1 2 3 4 5 6 7 8 0

Sample Output

plain text

Solution found:

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Result

Thus, the 8-puzzle problem was solved successfully using the BFS algorithm

```
        if state == goal:
            print ("Solution found:")
            for step in path:
                for i in range (0, 9, 3):
                    print (step [i:i+3])
                print ( )
            return

    zero = state.index (0)
    row, col = divmod (zero, 3)

    for dr, dc in moves:
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            new_state = state [:]
            new_state [zero], new_state[new_pos] = (
                new_state [new_pos], new_state [zero]
            )

            key = tuple (new_state)
            If key not in visited:
                visited.add (key)
                queue.append (new_state, path + [new_val])

    print ("No solution found.")

start = list (map (int, input ("Enter initial state: ").split()))
goal = list (map(int, input ("Enter goal state:").split()))

solve_puzzle(start, goal).
```